

HeterMM: applying in-DRAM index to heterogeneous memory-based key-value stores

Yunhong JI¹, Wentao HUANG², Xuan ZHOU (✉)³

¹ School of Information, Renmin University of China, Beijing 100872, China

² School of Computing, National University of Singapore, Singapore 117417, Singapore

³ School of Data Science and Engineering, East China Normal University, Shanghai 200062, China

© Higher Education Press 2024

1 Introduction

Emerging byte-addressable storage technologies, such as NVM (Non-Volatile Memory), provide a more cost-effective and larger-capacity alternative to DRAM (Dynamic Random Access Memory) [1], presenting new opportunities to address the high cost, limited capacity, and volatility of in-memory key-value (KV) stores. Numerous efforts have been dedicated to redesigning conventional structures on NVM. However, they were challenged by the substantial engineering cost and increased complexity to be integrated into existing systems. Thus, a general framework to apply existing indexes to KV stores on NVM becomes more attractive [2].

In this paper, we propose a general framework named HeterMM, for heterogeneous memory architecture consisting of DRAM and NVM. It is designed to fully leverage the superior performance of DRAM, and make the performance of the system as close to the in-DRAM one as possible. The specific contributions of this work are as summarized follows:

- 1) We designed HeterMM, a general framework that applies mature and well-designed volatile indexes to heterogeneous memory-based KV stores while reserving their strengths. It utilizes an operation log mechanism to ensure persistence and facilitate failure recovery.
- 2) HeterMM tries to achieve performance as close to that of in-DRAM systems as possible by maintaining both the index and hot data in DRAM. Moreover, it employs an efficient storage mechanism for heterogeneous memory, including an adaptable read cache in DRAM and an append-only storage mechanism on NVM.
- 3) We combined HeterMM with different Hash and B+ tree indexes, and evaluating them using the YCSB (Yahoo! Cloud Serving Benchmark) [3]. The evaluation demonstrates that HeterMM outperforms leading conversion frameworks and NVM-optimized designs across various workloads.

2 System design

As depicted in Fig. 1, HeterMM is composed of a plugged-in and in-DRAM index, a data storage mechanism on heterogeneous memory, and an operation log on NVM. In particular, the index, which is the most frequently accessed and typically in a small unit and random order, is not friendly to NVM [4]. Besides, their data structures, which are usually optimized for DRAM, may not perform as effectively on NVM. Therefore, we remain the index in DRAM.

In HeterMM, newly written data resides in DRAM, and old data is flushed to NVM in batches. Each data is allocated a logical address upon its arrival, which remains the same unless the data is updated out-of-place. In particular, the persistence of NVM can ensure the durability of data residing in it, while an operation log is applied to ensure the durability of data residing in DRAM. Firstly, data in DRAM is updated in place, which could be regarded as early compaction and reduces data volume flushed to NVM. Secondly, data in NVM can be regarded as a checkpoint which can be used to cut off the operation log. Moreover, to optimize access to read-only data in NVM, we divide the DRAM region into a read cache and a write region, with the former holding frequently accessed data residing in NVM while the latter holding newly arrived data. They share the same space in DRAM and can be resized dynamically according to the workload.

2.1 Plug-in programming model

HeterMM offers a set of plug-in APIs (Application Programming Interfaces), as depicted in Fig. 2. In particular, the *set_value* is responsible for storing the data (in DRAM or NVM) and returning a unique and increasing logical address to the caller. The logical address replaces the value in the original index and is used when calling the *get_value* to get the actual position of the data during a *get* request. Figure 3 illustrates how to utilize them to modify an in-DRAM hash table. The only changes required are the replacement of the value *get/set* codes with the provided APIs.

2.2 Memory management

As Fig. 4 shows, HeterMM manages DRAM and NVM as a

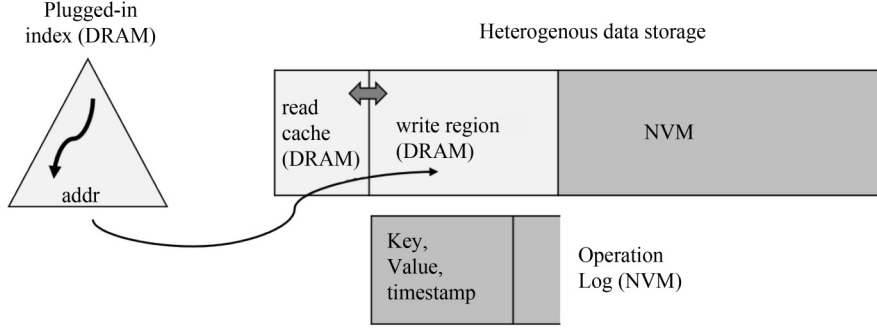


Fig. 1 Overview of the HeterMM

```

1 Value_t* get_value(LogAddr addr);
2 LogAddr set_value(LogAddr addr, Key_t k, Value_t* v);

```

Fig. 2 APIs provided by HeterMM

```

1 int hash_set(Hash* h, Key_t key, Value_t value){
2     {bkt, idx} = get_bkt(h, key);
3     bkt[idx].key = key;
4     #ifndef USE_HeterMM //The original code
5         bkt[idx].value = value;
6     #else // Applying HeterMM
7         bkt[idx].value = set_value(NULL, key, &value);
8     #endif
9 }
10
11 Value_t* hash_get(Hash* h, Key_t key){
12     {bkt, idx} = get_bkt(h, key);
13     #ifndef USE_HeterMM //The original code
14         return &bkt[idx].value;
15     #else // Applying HeterMM
16         return get_value(bkt[idx].value);
17     #endif
18 }

```

Fig. 3 Simplified code snippet of the *set* and *get* functions when utilizing HeterMM on a hash table

unified flat-addressable space, which consists of three parts:

- 1) The **in-place-update region**, where data resides in DRAM and can be updated in place.
- 2) The **overlap region**, where data is in the process of flushing into the NVM and may reside in both DRAM and NVM, therefore must be updated out-of-place.
- 3) The **persisted region**, where data resides in NVM and must be updated out-of-place.

Newly arrived or out-of-place updated data are appended to the in-place-update region while old data will move on to the overlap region or persisted region asynchronously and

sequentially. As shown in Fig. 4, two markers, $off_{ReadOnly}$ and off_{DRAM} , are used to distinguish the three storage regions. They increase along with the insertion of new data. Specifically, when an update occurs on a cached tuple, an out-of-place update will be applied as the old data resides in NVM. Therefore, the cached entry becomes invalid and waits to be freed and reclaimed.

In particular, the overlap region works as the middle tier to distinguish hot and cold data by providing hot data a chance to keep residing in the DRAM. This is because the old but hot data in it is highly likely to be brought back to the in-place-update region before it reaches the persisted region. Therefore, all accesses to the hot data can be served by the DRAM.

Specifically, in the *get value* API, HeterMM retrieves the data d based on the provided logical address $addr$ as follows:

$$d = \begin{cases} pages[P_{no}] + (addr \bmod (2^k - 1)), & \text{if } addr \geq off_{DRAM}, \\ blocks[B_{no}] + (addr \bmod (2^m - 1)), & \text{if } addr < off_{DRAM}, \end{cases}$$

$$P_{no} = \frac{addr}{2^k} \bmod pageNum, \quad B_{no} = \frac{addr}{2^m}. \quad (1)$$

In Eq. (1), *pages* and *blocks* represent arrays holding base addresses of DRAM pages and NVM blocks. *pageNum* is the total number of pages in DRAM. When a page is allocated, its corresponding range of logical addresses is determined, and its base address is recorded in the *pages* array. A logical address will always map to a definite slot (defined by P_{no}) in the *pages* array unless the $off_{ReadOnly}$ grows to greater than it. More details are in Online Resource 1.

3 Evaluation

We conducted experiments with Intel® Optane™ Persistent Memory 200 Series with all NVM DIMMs configured in App Direct mode. Our evaluation was based on the YCSB [3] with

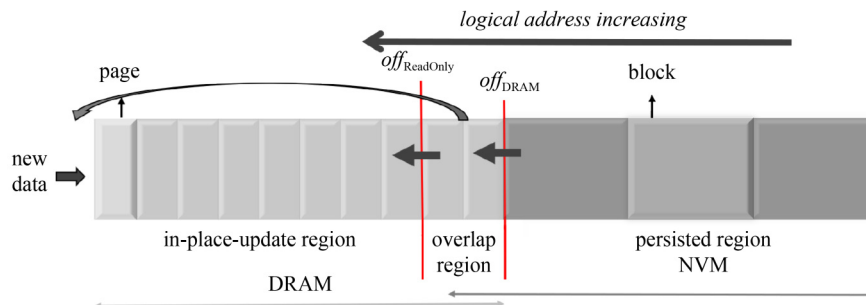


Fig. 4 The logical space unifying DRAM and NVM

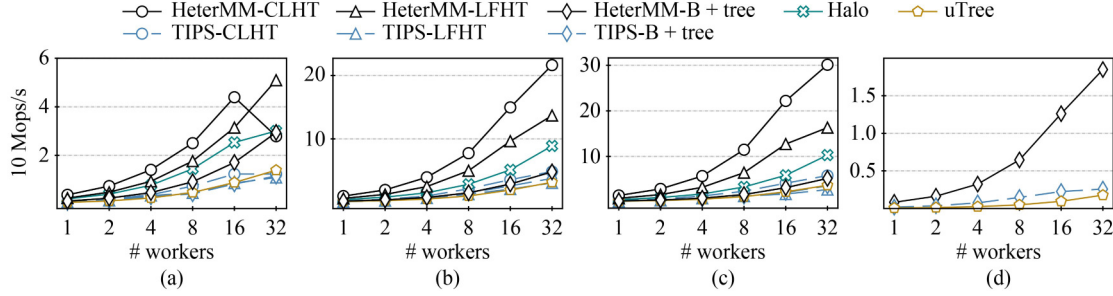


Fig. 5 Performance comparison among HeterMM-based, TIPS-based, and NVM-optimized KV stores, with an increasing number of workers and different ratios of read (R), update (U), and scan (S) requests. (a) 50%R, 50%U; (b) 95%R, 5%U; (c) 100%R; (d) 95%S, 5%U

200 million tuples consisting of 8-byte keys and 64-byte values. We combined HeterMM with a cache-line hash table (CLHT), a lock-free hash table (LFHT), and a B+ tree respectively, and allocated 4 GB of DRAM for the DRAM region, including the write region and the read cache.

We compared HeterMM with the following systems: 1) TIPS [2], which is a state-of-the-art general conversion framework. We used it to convert the same three indexes (i.e., CLHT, LFHT, B+ tree) into a persistent form; 2) Halo [4] and μ Tree [5], which are state-of-the-art hybrid DRAM and NVM-based hash tables and B+ trees, respectively.

Figure 5 presents the throughput of all comparison systems, which proves the superiority of HeterMM. Besides, we can make the following observations.

(a) HeterMM-based systems consistently outperformed TIPS-based ones when applying the same index. This can be attributed to the better utilization of DRAM and reduced access to NVM in HeterMM. Specifically, TIPS maintains several logs to ensure durability and crash consistency, including an operation log, an undo log, and a memory allocation log, which leads to write amplification on NVM. Besides, TIPS only caches the index in DRAM. Even if we configured it as caching 100% of the index in our experiments, all values in it reside in NVM. In contrast, hot data in HeterMM can be located at DRAM, resulting in fewer NVM reads.

(b) HeterMM-based systems could achieve better performance than NVM-optimized systems. The results demonstrate that the best performance among all comparison systems, across various workloads, is achieved by HeterMM-based systems. In particular, HeterMM is designed to minimize its overhead and make the system performance as close to a pure in-DRAM one as possible. Therefore, as long as a proper index is chosen, the HeterMM-based system is expected to achieve a better performance than others.

4 Conclusion

We propose HeterMM, a versatile framework that leverages

in-DRAM indexes in KV stores on heterogeneous memory. HeterMM incorporates a plug-in programming model, allowing for the integration of various types of indexes. By prioritizing the maintenance of both index and hot data in DRAM, HeterMM maximizes the utilization of the superior performance of DRAM. Our evaluation demonstrates that HeterMM outperforms existing state-of-the-art frameworks that convert in-DRAM indexes to persistent ones. Furthermore, HeterMM can surpass NVM-specific KV stores by carefully selecting the appropriate index for specific scenarios.

Acknowledgement This project was partially supported by a grant funded by the Ministry of Education (Singapore) (Title: inPMdb: An in-Persistent Memory Database System; WBS NO: A8000082-00-00) and Shanghai Engineering Research Center of Big Data Management.

Competing interests The authors declare that they have no competing interests or financial conflicts to disclose.

Electronic supplementary material Supplementary material is available in the online version of this article at journal.hep.com.cn and link.springer.com.

References

1. Huang W, Ji Y, Zhou X, He B, Tan K. A design space exploration and evaluation for main-memory hash joins in storage class memory. *Proceedings of the VLDB Endowment*, 2023, 16(6): 1249–1263
2. Ramanathan M K, Kim W H, Fu X, Monga S K, Lee H W, Jang M, Mathew A, Min C. TIPS: making volatile index structures persistent with DRAM-NVMM tiering. In: *Proceedings of 2021 USENIX Annual Technical Conference*. 2021, 773–787
3. Cooper B F, Silberstein A, Tam E, Ramakrishnan R, Sears R. Benchmarking cloud serving systems with YCSB. In: *Proceedings of the 1st ACM Symposium on Cloud Computing*. 2010, 143–154
4. Hu D, Chen Z, Che W, Sun J, Chen H. Halo: a hybrid PMem-DRAM persistent hash index with fast recovery. In: *Proceedings of 2022 International Conference on Management of Data*. 2022, 1049–1063
5. Chen Y, Lu Y, Fang K, Wang Q, Shu J. uTree: a persistent B+-tree with low tail latency. *Proceedings of the VLDB Endowment*, 2020, 13(12): 2634–2648